

MODUL PRAKTIKUM EKSKLUSIF (KERNEL HACKING)

TOPIK: MODIFIKASI KERNEL MIT xv6 & PENAMBAHAN SYSTEM CALL

Mata Kuliah: Sistem Operasi (INF-201) – Alokasi Waktu: 200 Menit – Bobot Evaluasi: 10%

Program Studi Informatika – Fakultas Teknik – Universitas Bengkulu

1. CAPAIAN PEMBELAJARAN MODUL

Setelah menyelesaikan modul tingkat lanjut ini, mahasiswa diharapkan mampu:

1. Melakukan kompilasi dan emulasi arsitektur sistem operasi menggunakan QEMU.
2. Memahami alur kerja eksekusi antara *User-Space* dan *Kernel-Space*.
3. Menambahkan *System Call* baru langsung ke dalam *source code* kernel sistem operasi xv6.

2. LANDASAN TEORI RINGKAS

Sistem operasi memisahkan memori menjadi dua wilayah: *Kernel-Space* yang memiliki akses penuh ke perangkat keras, dan *User-Space* yang sangat dibatasi. Agar program pengguna dapat meminta layanan kernel (seperti membaca file atau membuat proses), diperlukan jembatan yang disebut **System Call**. Pada arsitektur RISC-V, proses ini dilakukan melalui instruksi `ecall` yang akan memicu transisi *trap* ke dalam kernel.

3. PERSIAPAN LINGKUNGAN (ENVIRONMENT SETUP)

```
1 # 1. Instalasi dependensi RISC-V GNU Compiler dan QEMU
2 $ sudo apt-get install git build-essential gdb-multiarch qemu-system-misc
   gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
3
4 # 2. Kloning repositori MIT xv6
5 $ git clone https://github.com/mit-pdos/xv6-riscv.git
6 $ cd xv6-riscv
7
8 # 3. Jalankan Kernel OS dalam emulator QEMU
9 $ make qemu
```

Listing 1: Kloning dan Instalasi QEMU/xv6

4. TUGAS PRAKTIKUM: IMPLEMENTASI `sys_getreadcount()`

Tugas Anda adalah memodifikasi kernel xv6 agar dapat melacak berapa kali *system call* `read()` dipanggil secara global oleh semua proses di dalam sistem. Anda harus merancang system call baru bernama `getreadcount` yang mengembalikan nilai counter tersebut.

4.1 Langkah Kerja Modifikasi Kernel

Mahasiswa harus melakukan injeksi kode pada file-file internal kernel berikut:

1. `user/user.h`: Tambahkan prototipe fungsi C untuk *user-space*.
2. `user/usys.pl`: Tambahkan *entry* perl script agar Makefile dapat men-generate instruksi *assembly* `ecall` untuk `getreadcount`.

3. `kernel/syscall.h`: Definisikan konstanta `*system call number*` (misal: `#define SYS_getreadcount 22`).
4. `kernel/syscall.c`:
 - Deklarasikan array pointer ke eksternal fungsi `sys_getreadcount`.
 - Modifikasi fungsi internal `sys_read()` untuk meng-inkrementasi variabel global counter (gunakan `*spinlock*` untuk mencegah `*Race Condition*` di kernel!).
5. `kernel/sysfile.c`: Tulis implementasi aktual fungsi `uint64 sys_getreadcount(void)` yang me-return nilai counter.

5. KODE PENGUJIAN USER-SPACE

Buat program user-space baru bernama `count_test.c` di folder `user/` untuk menguji modifikasi kernel Anda. Tambahkan `_count_test` ke dalam `UPROGS` di file `Makefile`.

```

1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user/user.h"
4
5 int main(void) {
6     int count1 = getreadcount();
7     printf("Jumlah pemanggilan read() sebelum dites: %d\n", count1);
8
9     // Panggil fungsi read() secara sengaja
10    char buf[10];
11    read(0, buf, 1); // Membaca 1 byte dari stdin
12
13    int count2 = getreadcount();
14    printf("Jumlah pemanggilan read() setelah dites: %d\n", count2);
15
16    if (count2 > count1) {
17        printf("[OK] BERHASIL: System call getreadcount bekerja dengan baik\n");
18    } else {
19        printf("[FAILED] GAGAL: Counter tidak bertambah.\n");
20    }
21
22    exit(0);
23 }
```

Listing 2: Program Uji (`user/count_test.c`)

6. RUBRIK PENILAIAN

Indikator Penilaian	Bobot	Kriteria Lulus
Kompilasi xv6	20%	xv6 berhasil di- <i>*build*</i> ulang tanpa error syntax kernel.
Implementasi System Call	40%	Program <code>count_test</code> berhasil dieksekusi tanpa error <i>unknown sys call</i> .
Keamanan Kernel (Spinlock)	20%	Counter dilindungi oleh struktur <code>spinlock</code> untuk mencegah korupsi memori kernel.
Pemahaman Konseptual	20%	Mampu menjawab sesi wawancara asisten mengenai perbedaan <code>ecall</code> dan interrupt biasa.